

LẬP TRÌNH JAVA



Giảng viên: ThS Huỳnh Vĩnh Phát

Biên soạn: TS Trần Duy Thanh

Bài Học

Các khái niệm về Hướng đối tượng



Nội dung bài học

- ❖ So sánh lập trình truyền thống và hướng đối tượng
- ❖ Các khái niệm về lớp và đối tượng:
 - Đối tượng
 - Lớp
 - Package
 - Tính trừu tượng
 - Tính đóng gói
 - Tính kế thừa

Lập trình truyền thống

☐ **Phương pháp tiếp cận của lập trình truyền thống**

- ✓ Lập trình tuyến tính.
- ✓ Lập trình cấu trúc

☐ **Ưu điểm**

- ✓ Chương trình rõ ràng, dễ hiểu, dễ theo dõi.
- ✓ Tư duy giải thuật rõ ràng.

☐ **Khuyết điểm**

- ✓ Không hỗ trợ việc sử dụng lại mã nguồn.
- ✓ Khi thay đổi cấu trúc dữ liệu, phải thay đổi giải thuật.
- ✓ Phải giải quyết các mối quan hệ vĩ mô giữa các module phần mềm trong các dự án lớn.

Lập trình hướng đối tượng

☐ Đặc điểm cơ bản

- ✓ Tập trung vào dữ liệu thay cho các hàm.
- ✓ Chương trình được chia thành các đối tượng độc lập.
- ✓ Cấu trúc dữ liệu được thiết kế sao cho đặc tả được các đối tượng.
- ✓ Dữ liệu được che giấu, bao bọc.
- ✓ Các đối tượng trao đổi với nhau thông qua các hàm.
- ✓ Chương trình được thiết kế theo hướng tiếp cận từ dưới lên:

➔ Là cách lập trình tập chung vào xung quanh đối tượng, khi đó dữ liệu được người dùng quan tâm là chính, dữ liệu luôn luôn được quản lý mà không tự do hoạt động trong chương trình. 1 chương trình khi đó được phân tích thành các đối tượng nhỏ hơn để dễ quản lý.

Lập trình hướng đối tượng

☐ Một số ưu điểm nổi bật

- ✓ Không có nguy cơ dữ liệu bị thay đổi tự do trong chương trình.
- ✓ Khi thay đổi cấu trúc dữ liệu của một đối tượng, không cần thay đổi mã nguồn của các đối tượng khác.
- ✓ Có thể sử dụng lại mã nguồn, tiết kiệm tài nguyên.
- ✓ Phù hợp với các dự án phần mềm lớn, phức tạp.

Khái niệm Lớp và Đối tượng

- ❖ Khái niệm **đối tượng** (object) trong lập trình hướng đối tượng giống như một đối tượng cụ thể trong thế giới thực.
- ❖ Mỗi đối tượng có các thuộc tính và các hành vi riêng.
 - **Thuộc tính** (attribute) mô tả đặc điểm của đối tượng.
 - Hành vi là phương thức hoạt động của đối tượng, gọi tắt là **phương thức** (method).

Khái niệm Lớp và Đối tượng

❖ Ví dụ: Phân số

❖ Đặc điểm

- Tử số
- Mẫu số

❖ Thao tác

- Cộng, trừ, nhân, chia
- Tối giản
- Nghịch đảo

Khái niệm Lớp và Đối tượng

❖ Ví dụ: xe hơi

- Màu trắng
- 4 cửa
- 4 bánh
- Hiệu Toyota
- Chạy tới
- Chạy lui
- Xe dừng
- ...

Khái niệm Lớp và Đối tượng

❖ Đối tượng:

❖ XeHoi

❖ Hiệu xe

❖ Màu xe

❖ Số bánh xe

❖ Số cửa

❖ Chạy tới

❖ Chạy lui

❖ Dừng xe

❖ Tên đối tượng

❖ Thuộc tính

❖ Phương thức

Khái niệm Lớp và Đối tượng

- ❖ Các đối tượng có các đặc điểm (thuộc tính và phương thức) giống nhau được gom nhóm thành một lớp để phân biệt với các đối tượng khác và dễ quản lý.
- ⇒ Một **lớp** (class) là sự phân loại của các đối tượng hay là kiểu (type) của đối tượng.
- ❖ Ví dụ:
 - Các chiếc xe Toyota, Honda, Porsche thuộc lớp xe hơi.
 - Các con chó giữ nhà, chó săn, chó kiểng thuộc lớp chó.

Khái niệm Lớp và Đối tượng

- ❖ Như vậy **Lớp** là một khái niệm trừu tượng, dùng để chỉ một tập hợp các đối tượng có mặt trong hệ thống.
- ❖ **Lớp** có thuộc tính và phương thức:
 - Thuộc tính của lớp tương ứng với thuộc tính của đối tượng.
 - Phương thức của lớp tương ứng với các hành động của đối tượng.

Khái niệm Lớp và Đối tượng

- ❖ Một **Lớp** có thể có một trong các khả năng sau:
 - Hoặc chỉ có thuộc tính, không có phương thức.
 - Hoặc chỉ có phương thức, không có thuộc tính.
 - Hoặc có cả thuộc tính và phương thức, trường hợp này là phổ biến nhất.
- Lớp không có thuộc tính và phương thức nào là các lớp trừu tượng. Các lớp này không có đối tượng tương ứng.

Khái niệm Lớp và Đối tượng

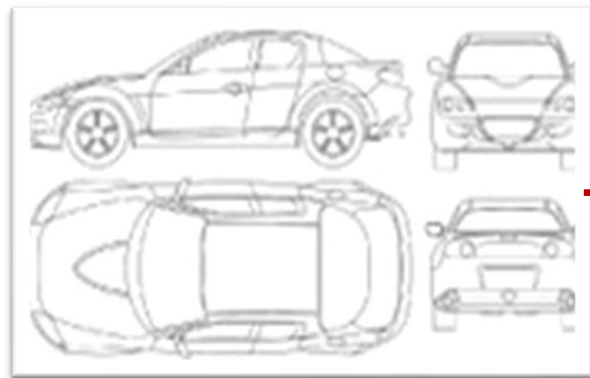
❖ Gói (package)

- Một nhóm các lớp (classes) và giao diện (interfaces) được tổ chức thành một đơn vị quản lý theo hình thức không gian tên gọi là package.
- Lợi ích của package là tổ chức sắp xếp lại hệ thống thông tin các lớp trong dự án một cách khoa học, giúp cho việc theo dõi bảo trì dự án được tốt nhất.

Khái niệm Lớp và Đối tượng

❖ Tính trừu tượng:

- Lớp (Class) là một khái niệm trừu tượng, đối tượng là một thể hiện cụ thể của lớp.
- Ví dụ:
 - Bản thiết kế của chiếc xe hơi là lớp.
 - Chiếc xe hơi được tạo ra từ bản thiết kế là đối tượng.



Class



Object

Khái niệm Lớp và Đối tượng

❖ Tính trừu tượng:

- Từ những đối tượng giống nhau: trừu tượng hóa thành một lớp:
- Chỉ đưa ra các thuộc tính và phương thức cần thiết của đối tượng trong lập trình.

❖ Ví dụ:



Khái niệm Lớp và Đối tượng

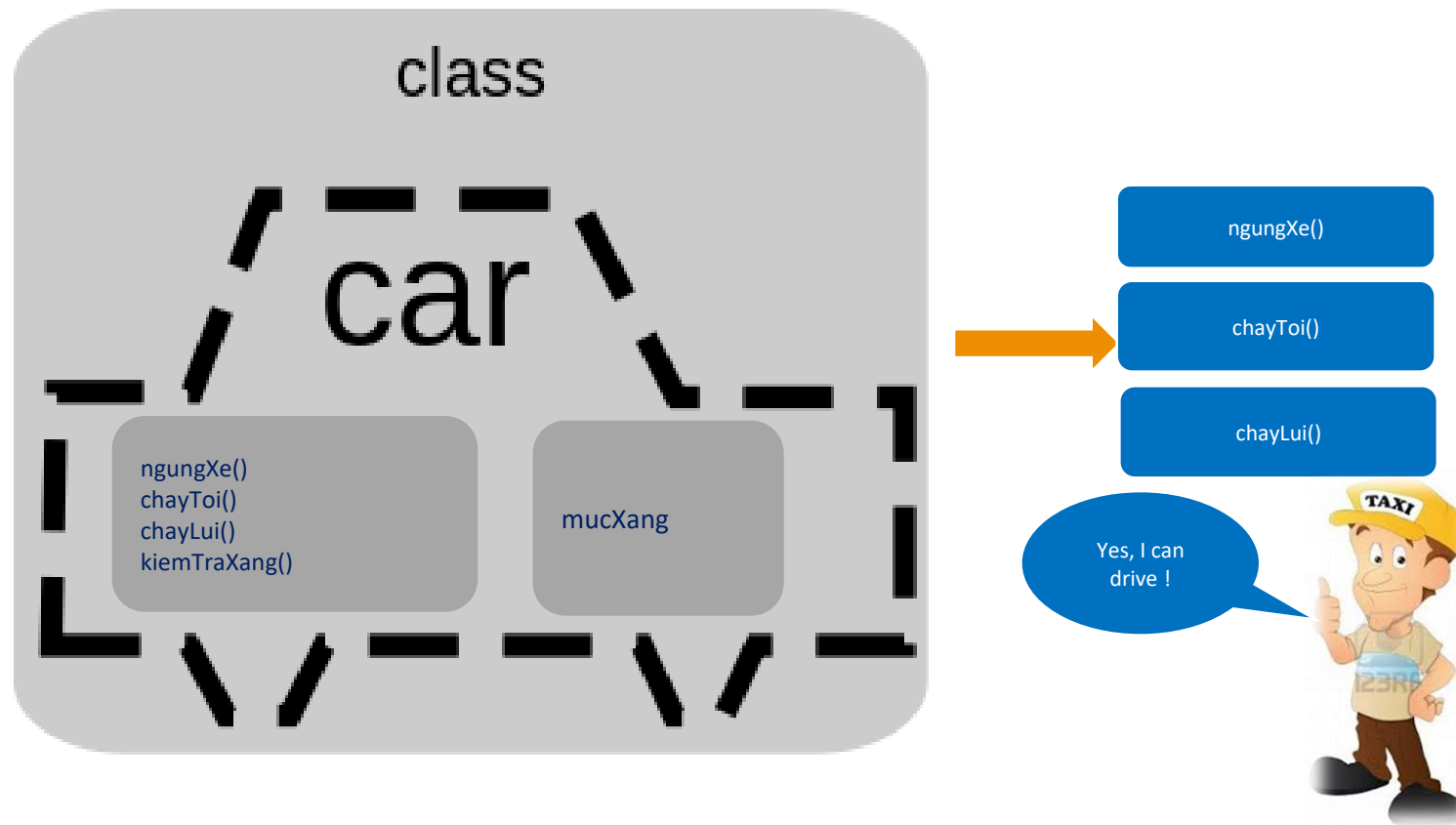
❖ Tính đóng gói:

- Mỗi lớp được xây dựng để thực hiện một nhóm chức năng đặc trưng của riêng lớp đó.
- Tất cả mọi thao tác truy xuất vào thành phần dữ liệu từ đối tượng này qua đối tượng khác phải được thực hiện bởi các phương thức (method) của chính đối tượng chứa dữ liệu.
- Tính đóng gói cho phép dấu thông tin của đối tượng bằng cách kết hợp thông tin và các phương thức liên quan đến thông tin trong đối tượng.

Khái niệm Lớp và Đối tượng

❖ Tính đóng gói:

❖ Ví dụ:



Khái niệm Lớp và Đối tượng

❖ Tính kế thừa:

- Cho phép xây dựng một lớp mới dựa trên các định nghĩa của một lớp đã có.
- Lớp đã có gọi là lớp Cha, lớp mới phát sinh gọi là lớp Con
- Lớp con kế thừa tất cả các thành phần của lớp Cha, có thể mở rộng các thành phần kế thừa và bổ sung thêm các thành phần mới.

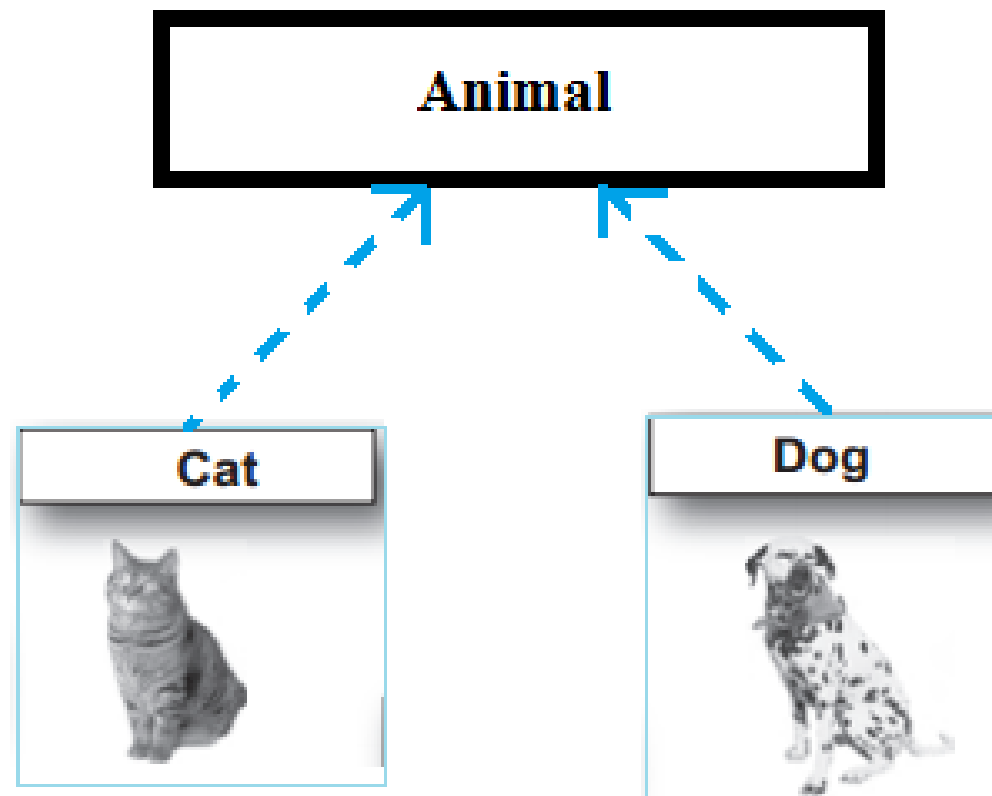


➤ Khái niệm này sẽ trình bày chi tiết ở chương sau

Khái niệm Lớp và Đối tượng

❖ Tính kế thừa:

❖ Ví dụ:



Bài Học

Các Quy tắc khai báo lớp



Nội dung bài học

- ❖ Quy tắc đặt tên Lớp trong java
- ❖ Quy tắc đặt tên Constructor
- ❖ Quy tắc đặt tên thuộc tính trong java, cách tạo getter/setter
- ❖ Quy tắc đặt tên phương thức

Quy tắc đặt tên Lớp trong java

❖ Qui tắc đặt tên lớp:

- ✓ Tên lớp nên là một danh từ
- ✓ Tên lớp có thể gồm nhiều từ, ký tự đầu tiên của mỗi từ nên viết hoa
- ✓ Tên lớp nên đặt đơn giản, dễ nhớ, và có ý nghĩa
- ✓ Tên lớp không được trùng với từ khóa của Java
- ✓ Tên lớp không thể bắt đầu bằng số, nhưng có thể bắt đầu bằng dấu '\$' và dấu gạch dưới '_'.

Quy tắc đặt tên Lớp trong java

❑ Lớp (Class)

Cách khai báo lớp

```
[Chỉ định từ truy xuất]    class    [tên lớp] {  
    [thuộc tính];  
    [phương thức];  
}
```

Trong đó:

- **Chỉ định từ truy xuất:**

	Lớp (Class)	Gói (Package)	Lớp con (Subclass)	Ngoài
public	Có	Có	Có	Có
protected	Có	Có	Có	Không
Không có (no modifier)	Có	Có	Không	Không
private	Có	Không	Không	Không

- **Thuộc tính:** mô tả các thông tin liên quan về class.
- **Phương thức:** mô tả các phương thức xử lý có thể làm việc được với class.

Quy tắc đặt tên Lớp trong java

```
public class SinhVien {  
    private String hoTen;  
    private double diem;  
    public void show() {  
        System.out.println(hoTen + ": " + diem);  
    }  
}
```

Quy tắc đặt tên Constructor

- ❖ Constructor là phương thức đặc biệt: Có tên trùng y xì với tên lớp, không có kiểu trả về.
- ❖ Dùng để khởi tạo các giá trị cho các thuộc tính khi một đối tượng được cấp phát bộ nhớ.
- ❖ Có 2 loại Constructor: Mặc định và có đối số.

Quy tắc đặt tên Constructor

- ❖ Constructor Mặc định: Dùng để tự động khởi tạo các giá trị cho các thuộc tính thường là các giá trị mặc định nào đó do chủ ý của lập trình viên.

```
public class SinhVien {  
    private String hoTen;  
    private double diem;  
    public SinhVien()  
    {  
        hoTen="Obama";  
        diem=9.5;  
    }  
}
```

Quy tắc đặt tên Constructor

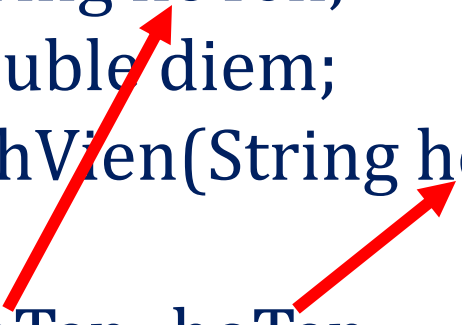
❖ Ví dụ:

```
SinhVien obama=new SinhVien();
```

Quy tắc đặt tên Constructor

- ❖ Constructor có đối số: Dùng để khởi tạo các giá trị cho các thuộc tính lúc tạo đối tượng, các giá trị này được truyền từ đối số tạo đối tượng.

```
public class SinhVien {  
    private String hoTen;  
    private double diem;  
    public SinhVien(String hoTen, double diem)  
    {  
        this.hoTen=hoTen;  
        this.diem=diem;  
    }  
}
```



Quy tắc đặt tên Constructor

❖ Ví dụ:

```
SinhVien obama=new SinhVien("Obama", 8.5);
```

Quy tắc đặt tên thuộc tính

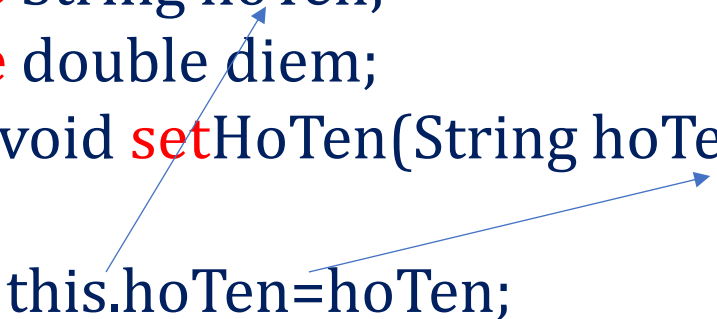
- ❖ Ký tự đầu tiên viết thường, ký tự đầu tiên của các từ tiếp theo viết HOA
- ❖ Để **private** nếu không muốn cho kế thừa
- ❖ Để **protected** nếu muốn cho kế thừa
- ❖ Tuyệt đối không được để public

```
public class SinhVien {  
    private String hoTen;  
    private double diem;  
}
```

Quy tắc đặt tên thuộc tính

- ❖ Ta cần xây dựng các getter/setter để truy suất giá trị các thuộc tính:

```
public class SinhVien {  
    private String hoTen;  
    private double diem;  
    public void setHoTen(String hoTen)  
    {  
        this.hoTen=hoTen;  
    }  
    public String getHoTen()  
    {  
        return this.hoTen;  
    }  
}
```



Tương tự cho thuộc tính diem

Quy tắc đặt tên thuộc tính

- ❖ Các đối tượng ở ngoài không được phép truy suất trực tiếp vào các thuộc tính bên trong lớp, chỉ được phép tương tác thông qua getter / setter:

SinhVien sv=**new** SinhVien("Obama", 8.5);

Lấy tên: sv.getTen();

Đổi tên: sv.setTen("Putin");

Đổi điểm: sv.setDiem(9.5);

Lấy điểm: sv.getDiem();

Quy tắc đặt tên phương thức

- ❖ Phương thức là các hành động(nghệp vụ) trên từng đối tượng. NÓ là tập các giải thuật để giải quyết một công việc cụ thể nào đó.
- ❖ Phương thức có kiểu trả về hoặc void, có các đối số truyền vào.
- ❖ Ký tự đầu tiên viết thường, ký tự đầu tiên của các từ tiếp theo viết HOA:

Quy tắc đặt tên phương thức

```
public class SinhVien {  
    private String hoTen;  
    private double diem;  
    public void xuấtThôngTin()  
    {  
        //làm cái gì đó.  
    }  
}
```

```
SinhVien sv=new SinhVien("Obama", 8.5);  
sv.xuatThongTin();
```

Bài Học

Các loại phương thức



Nội dung bài học

- ❖ Service method
- ❖ Support method

Service method & Support Method

- ❖ Service Method là các phương thức **public** để cung cấp ra ngoài cho các đối tượng sử dụng
- ❖ Support Method là các phương thức **private** dùng để hỗ trợ cho các Service method. Các đối tượng ở ngoài không thể truy suất tới các Support Method

Ví dụ:

```
public class TamGiac {  
    private double canhA;  
    private double canhB;  
    private int canhC;  
    public double chuVi()  
    {  
        return canhA+canhB+canhC;  
    }  
    private double nuaChuVi()  
    {  
        return chuVi()/2;  
    }  
    public double dienTich()  
    {  
        double p=nuaChuVi();  
        return Math.sqrt(p*(p-canhA)*(p-canhB)*(p-canhC));  
    }  
}
```

Bài Học

Overloading method



Nội dung bài học

- ❖ Khái niệm về Overloading Method
- ❖ Lợi ích của Overloading Method
- ❖ Cách cài đặt Overloading Method
- ❖ Cách sử dụng Overloading Method
- ❖ Parameter list

Overloading Method

- ✓ Là đặc điểm trong cùng 1 lớp có nhiều phương thức cùng tên nhưng khác nhau về Signature.
- ✓ Signature bao gồm: Số lượng các đối số hoặc kiểu dữ liệu các đối số hoặc thứ tự các đối số.
- ✓ Kiểu dữ liệu trả về không được tính vào signature
- ✓ Lợi ích của Overloading là **khả năng tái sử dụng lại phương thức và giúp việc gọi hàm “uỷển chuyển”**.
- ✓ Các Constructor là trường hợp đặc biệt của Overloading Method

Overloading Method

```
float tryMe(int x)
{
    return x + .375;
}
```

Invocation

```
result = tryMe(25, 4.32)
```

```
float tryMe(int x, float y)
{
    return x*y;
}
```



Overloading Method

✓ Ví dụ Tái sử dụng Constructor:

```
public class SinhVien {  
    private String hoTen;  
    private double diem;  
    public SinhVien(String hoTen)  
    {  
        this.hoTen=hoTen;  
    }  
    public SinhVien(String hoTen, double diem)  
    {  
        this(hoTen) ;  
        this.diem=diem;  
    }  
}
```

Parameter list

- ❖ Java cung cấp một loại phương thức đặc biệt đó là Parameter List, cũng là một trường hợp đặc biệt của Overloading Method.

```
public int Sum(int ...arr)
{
    int s = 0;
    for(int x in arr)
    {
        s += x;
    }
    return s;
}
```

Sum(1, 2, 4)
Sum()
Sum(1, 5, -8, 2)

- ❖ Ta có thể truyền bao nhiêu đối số kiểu int vào cho phương thức Sum cũng được.

Bài Học

Tham chiếu this



Nội dung bài học

- ❖ Instance variable và local variable
- ❖ this là gì?
- ❖ Ứng dụng của this

Instance variable và local variable

- ❖ **Instance variable** là các thuộc tính, các biến khai báo ngoài lớp. Toàn bộ các hàm trong lớp đều có thể truy xuất được.
- ❖ **Local variable** là các biến được khai báo trong đối số của hàm hoặc nội dung hàm. Chỉ có hàm này mới truy xuất được các biến local variable của nó

Instance variable và local variable

```
public class SinhVien {  
    private String hoTen;  
    private double diem;  
    public SinhVien(String hoTen)  
    {  
        this.hoTen=hoTen;  
    }  
    public SinhVien(String hoTen, double diem)  
    {  
        this(hoTen) ;  
        int x=6;  
        this.diem=diem;  
    }  
}
```

this là gì?

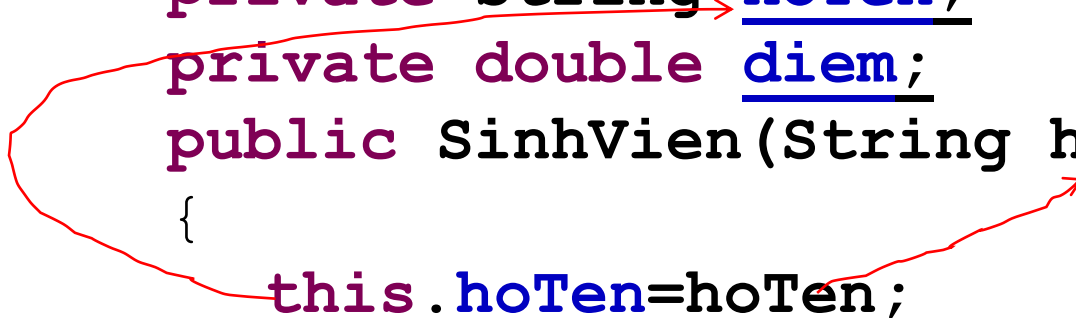
❖ Là đối tượng hiện tại trong 1 lớp

```
public class SinhVien {  
    private String hoTen;  
    private double diem;  
    public SinhVien(String hoTen)  
    {  
        this.hoTen=hoTen;  
    }  
    public SinhVien(String hoTen, double diem)  
    {  
        this(hoTen) ;  
        this.diem=diem;  
    }  
}
```

Ứng dụng của this

- ❖ **this** thường dùng để truy suất trực tiếp tới các biến instance. Dùng để phân biệt việc truy suất instace và local khi 2 biến này trùng tên

```
public class SinhVien {  
    private String hoTen;  
    private double diem;  
    public SinhVien(String hoTen)  
    {  
        this.hoTen=hoTen;  
    }  
}
```



Bài Học

Alias và cơ chế gom rác tự động



Alias và cơ chế gom rác tự động

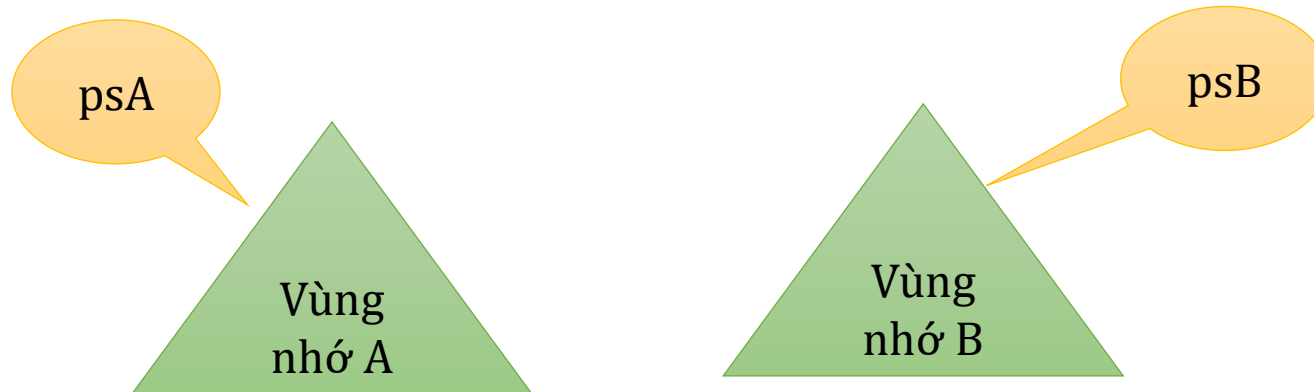
❖ Alias là đặc điểm mà trên một ô nhớ có nhiều biến đối tượng cùng trỏ tới.

❖ Ví dụ:

```
PhanSo psA=new PhanSo(1,5);
```

```
PhanSo psB=new PhanSo(3,7);
```

Lúc này trên thanh RAM sẽ có 2 ô nhớ cấp phát cho 2 đối tượng phân số được quản lý bởi 2 biến đối tượng psA và psB

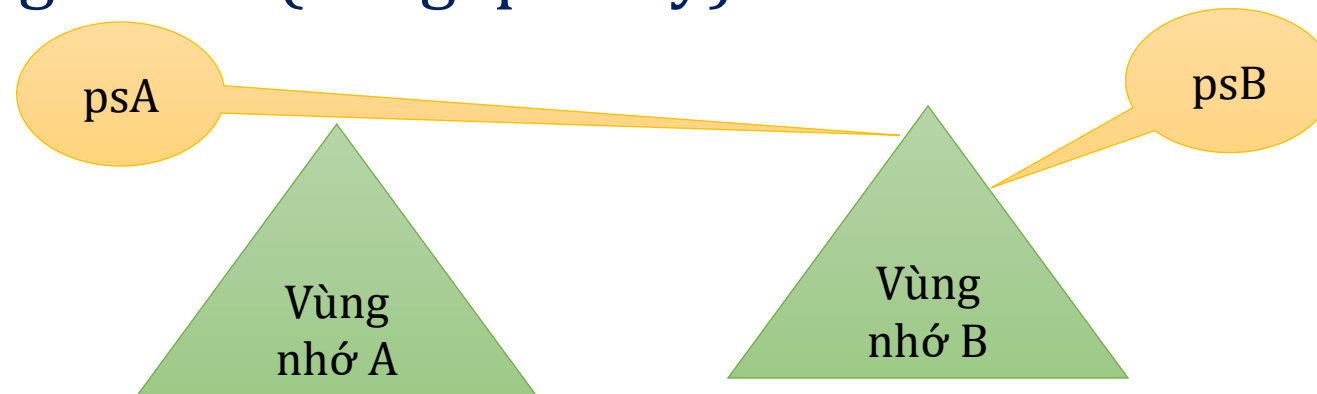


Alias và cơ chế gom rác tự động

❖ Giả sử ta thực hiện lệnh:

$psA = psB;$

➔ Ngôn ngữ nói “Phân số A bằng Phân số B”, nhưng hệ thống máy tính sẽ làm việc theo cơ chế “Phân số A trở tới vùng nhớ mà phân số B đang quản lý”. Hay nói cách khác “Vùng nhớ B” bây giờ có 2 biến đối tượng cùng trở tới (cùng quản lý)



Alias và cơ chế gom rác tự động

- ❖ Như vậy đã xuất hiện Alias ở “vùng nhớ B”. Lúc này sẽ xảy ra 2 hiện tượng như sau:
 - ❖ Tại “vùng nhớ B”, nếu psA thay đổi thông tin sẽ làm cho psB thay đổi thông tin (vì cả 2 đối tượng này cùng quản lý một vùng nhớ)
 - ❖ “Vùng nhớ A” không còn đối tượng nào tham chiếu tới, lúc này hệ thống sẽ tự động thu hồi bộ nhớ (hủy vùng nhớ A đã cấp trước đó), cơ chế này gọi là cơ chế gom rác tự động



Alias và cơ chế gom rác tự động

❖ Đôi khi trong quá trình thực hiện phần mềm ta có nhu cầu sao chép đối tượng ra (tạo thêm một đối tượng giống y xì đối tượng cũ nhưng nằm ở ô nhớ khác, để ta có thể tự do thay đổi thông tin trên đối tượng sao chép mà không làm ảnh hưởng tới đối tượng gốc). Java hỗ trợ chúng ta hàm clone trong interface **Cloneable** để sao chép đối tượng.

```
public class PhanSo implements Cloneable {  
    public PhanSo copy()  
        throws CloneNotSupportedException  
    {  
        return (PhanSo) this.clone();  
    }  
}
```

Alias và cơ chế gom rác tự động

❖ Ví dụ:

```
PhanSo psB = new PhanSo(1,4);
```

psB

Vùng
nhớ B

```
PhanSo psA = psB.copy();
```

psA

Sao chép toàn bộ thông tin trong Vùng nhớ B vào vùng nhớ A → Tức là ta có 2 đối tượng có thông tin giống nhau y xì nhưng nằm trên 2 ô nhớ hoàn toàn khác nhau

Vùng
nhớ A

psA thay đổi không ảnh hưởng gì tới **psB** và ngược lại

Bài Học

Các thành phần static



Nội dung bài học

❖ static và ý nghĩa sử dụng

Nội dung bài học

- ❖ Trong java có hỗ trợ khai báo static cho thuộc tính và phương thức.
- ✓ static cho thuộc tính: Gọi là Class Member
- ✓ static cho phương thức: Gọi là Class Method
- ❖ Muốn truy suất tới các static ta sẽ lấy trực tiếp tên lớp chấm thuộc tính/ phương thức mà không cần khởi tạo đối tượng.
- ❖ Các thành phần static sẽ được khởi tạo trước khi đối tượng được khởi tạo. Thường dùng để chia sẻ 1 vùng ô nhớ, các đối tượng sẽ sử dụng chung vùng này → chỉ cần đổi giá trị của thành phần static thì toàn bộ các đối tượng khác cũng thấy sự thay đổi này.

Nội dung bài học

❖ Ví dụ 1- biến static:

```
public class Class1 {  
  
    private static int value;  
    public void setValue(int x)  
    {  
        value=x;  
    }  
    public int getValue()  
    {  
        return value;  
    }  
}
```

Nội dung bài học

❖ Ví dụ 1- biến static:

```
public static void main(String[] args) {  
    Class1 c1=new Class1();  
    c1.setValue(9);  
    Class1 c2=new Class1();  
    System.out.println(c2.getValue());  
    c2.setValue(100);  
    System.out.println(c1.getValue());  
}
```

Kết quả:

9

100

Nội dung bài học

❖ Ví dụ 2 – Hàm static

```
public class Class2 {  
  
    public static void fn1(int x)  
    {  
        System.out.println(x);  
    }  
    public void fn2(int x)  
    {  
        System.out.println(x);  
    }  
}
```

Nội dung bài học

❖ Ví dụ 2 – Hàm static

```
public static void main(String[] args) {  
    Class2 c=new Class2();  
    c.fn2(113);  
  
    Class2.fn1(113);  
  
}
```

Bài Học

Khái niệm về kế thừa, lợi ích của kế thừa

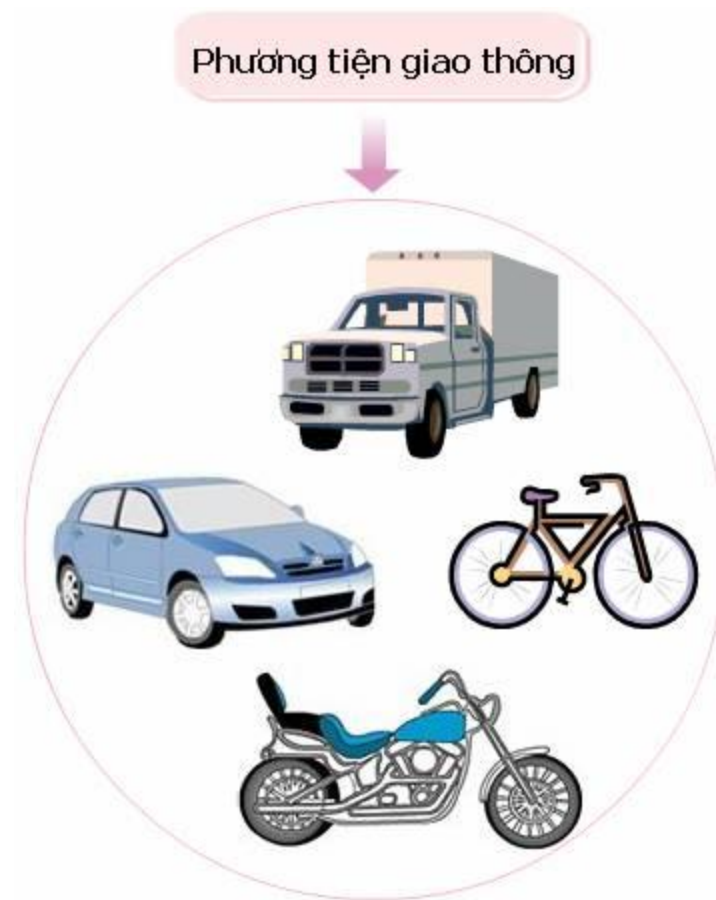
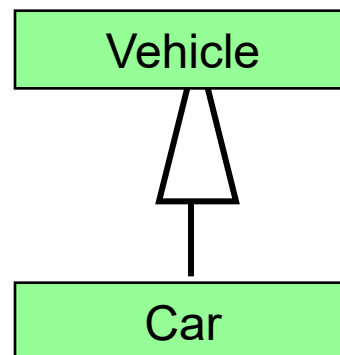


Kế thừa

❖ Các đối tượng có cùng chung một số đặc điểm, hành vi được nhóm lại với nhau

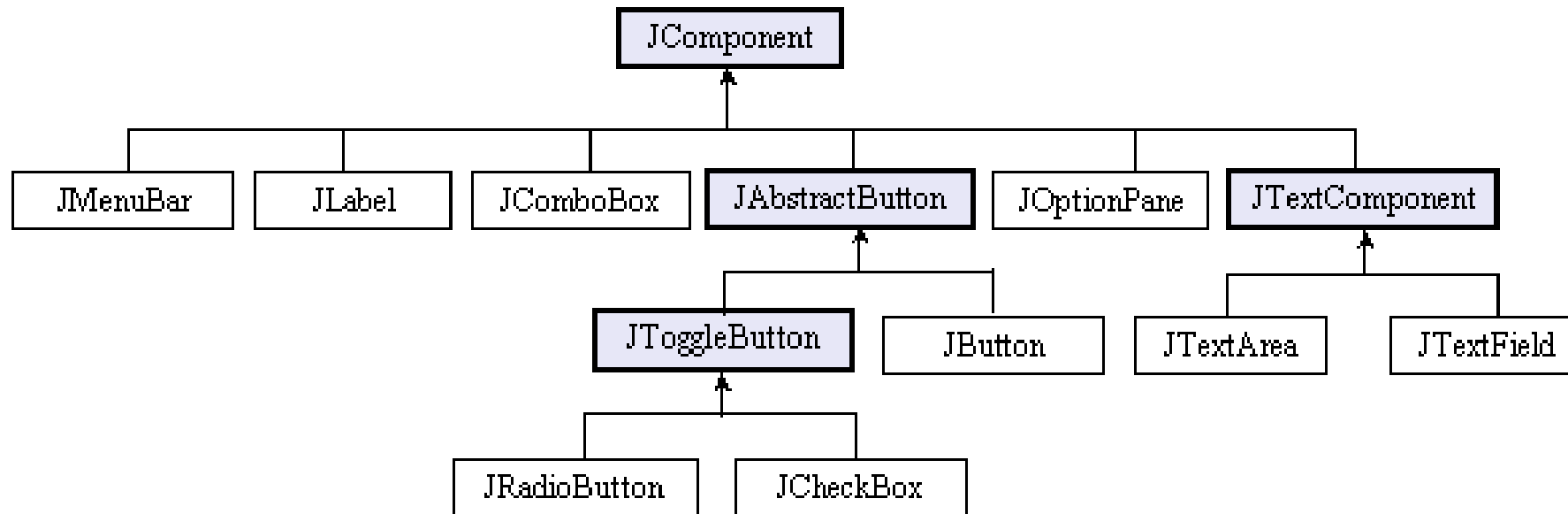
❖ Ví dụ

- Xe đạp
- Xe máy
- Xe hơi
- Xe tải
- ➔ Phương tiện giao thông



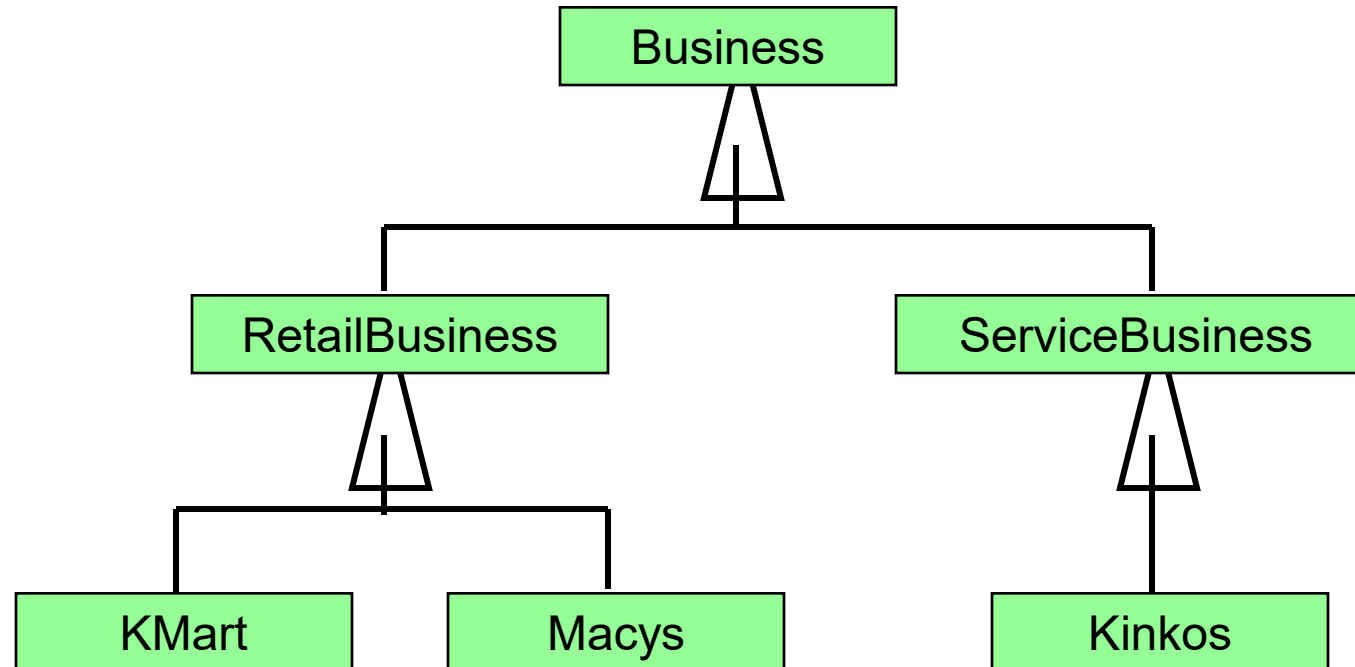
Kế thừa

❖ Ví dụ: Mô hình kế thừa Control trong Java



Kế thừa

❖ Ví dụ: Một lớp con có thể là lớp cha của các lớp khác



Kế thừa

- ❖ Tạo ra các lớp mới từ việc sử dụng lại những thành phần của lớp đã có
- ❖ Lợi ích
 - Nhất quán
 - Thuận tiện
 - Tái sử dụng code



Kế thừa

Trong 1 mô tả dự án thực tế làm sao
phát hiện được mô hình lớp mà ở đó
có tính kế thừa?

Cần nắm được khái niệm
Tổng quát hóa? chuyên biệt hóa?

Tổng quát hoá và chuyên biệt hóa

- ❖ Tổng quát hoá: Những đặc điểm chung mà các lớp đều có
- ❖ Chuyên biệt hóa: Những đặc điểm riêng chỉ có các lớp con mới có

Ví dụ: Nhân viên mọi người đều có mã, tên.
Nhưng Nhân viên chính thức và nhân viên thời vụ thì khác nhau ít nhất là cách tính lương.

Bài Học

Kỹ thuật viết Kế thừa từ Class



Nội dung bài học

- ❖ Lớp cơ sở (base class): làm cơ sở để các lớp khác kế thừa
- ❖ Lớp nhận (derived class): kế thừa đặc điểm của lớp cơ sở
- ❖ Khai báo
 - class LopNhan **extends** LopCoSo
 - Ví dụ:
 - class XeDap **extends** PhuongTienGiaoThong
 - class XeMay **extends** PhuongTienGiaoThong

Constructor

- ❖ Không được kế thừa
- ❖ Lớp con truy cập bằng từ khóa **super**

```
class HìnhHoc
{
    ...
    public HìnhHoc(double chuVi, double dienTich)
    {
        ChuVi = chuVi;
        DienTich = dienTich;
    }
}
class HìnhTron extends HìnhHoc
{
    ...
    public HìnhTron(double chuVi, double dienTich) {
        super(chuVi, dienTich);
    }
}
```

Abstract class

- ❖ Lớp Abstract là một lớp trừu tượng, không thể khởi tạo nó bằng toán tử **new**.
- ✓ Trong lớp có tồn tại phương thức abstract thì lớp đó cũng được định nghĩa là abstract.
- ✓ Phương thức trừu tượng là phương thức chỉ có định nghĩa tên hàm, các đối số (không có nội dung hàm)
- ✓ Bất cứ class nào kế thừa abstract class nào đó phải định nghĩa lại các abstract methods của lớp mà nó thừa kế hoặc không định nghĩa lại nhưng phải ghi lại abstract methods đó.

Abstract class

```
abstract class A{  
    abstract void method_1();  
}  
public class B extends A{  
    @Override  
    public void method_1(){  
        // cài đặt chi tiết cho phương thức method_1  
        // trong lớp con B.  
    }  
}  
public class C extends A{  
    @Override  
    public void method_1(){  
        // cài đặt chi tiết cho phương thức method_1  
        // trong lớp con C.  
    }  
}
```

Bài Học

Kỹ thuật viết Kế thừa từ Interface



interface

- ✓ Nó không phải là giao diện người dùng.
- ✓ Để tránh đi tính phức tạp của đa thừa kế (multi-inheritance) trong lập trình hướng đối tượng, Java thay thế bằng interface.
- ✓ Một lớp có thể kế thừa từ nhiều interface với các lớp khác để thừa hưởng thêm vùng dữ liệu và phương thức của các interface này.
- ✓ Interface được khai báo như một lớp, với các thuộc tính là các hằng và các phương thức là các phương thức trừu tượng.
- ✓ Methods trừu tượng là các methods chỉ có phần header, không có phần body.
- ✓ Trong các lớp có cài đặt các interface ta phải tiến hành cài đặt cụ thể các phương thức này.
- ✓ Các class có thể implements các interface.

interface

```
interface DienTich {  
    public abstract void sHinhVuong(float a);  
    public abstract void sHinhChuNhat(float a, float b);  
}
```

```
interface ChuVi {  
    public abstract void cVHinhVuong(float a);  
  
    public abstract void cVHinhChuNhat(float a, float b);  
}
```

```
interface CongThuc extends DienTich, ChuVi {  
}
```

interface

```
class CongThucTinh implements CongThuc {  
    @Override  
    public void sHinhVuong(float a) {  
        System.out.println("Diện tích hình vuông là: " + a * a);  
    }  
    @Override  
    public void sHinhChuNhat(float a, float b) {  
        System.out.println("Diện tích hình chữ nhật là: " + a * b);  
    }  
    @Override  
    public void cVHinhVuong(float a) {  
        System.out.println("Chu vi hình vuông là: " + 4 * a);  
    }  
    @Override  
    public void cVHinhChuNhat(float a, float b) {  
        System.out.println("Chu vi hình chữ nhật là: " + 2 * (a + b));  
    }  
}
```

interface

```
public class JavaAndroidVn {  
    public static void main(String[] args) {  
        CongThucTinh x = new CongThucTinh();  
        x.sHinhVuong(5.0f);  
        x.cVHinhVuong(5.0f);  
        x.sHinhChuNhat(5.5f, 10.0f);  
        x.cVHinhChuNhat(5.5f, 10.0f);  
    }  
}
```

Bài Học

Overriding Method

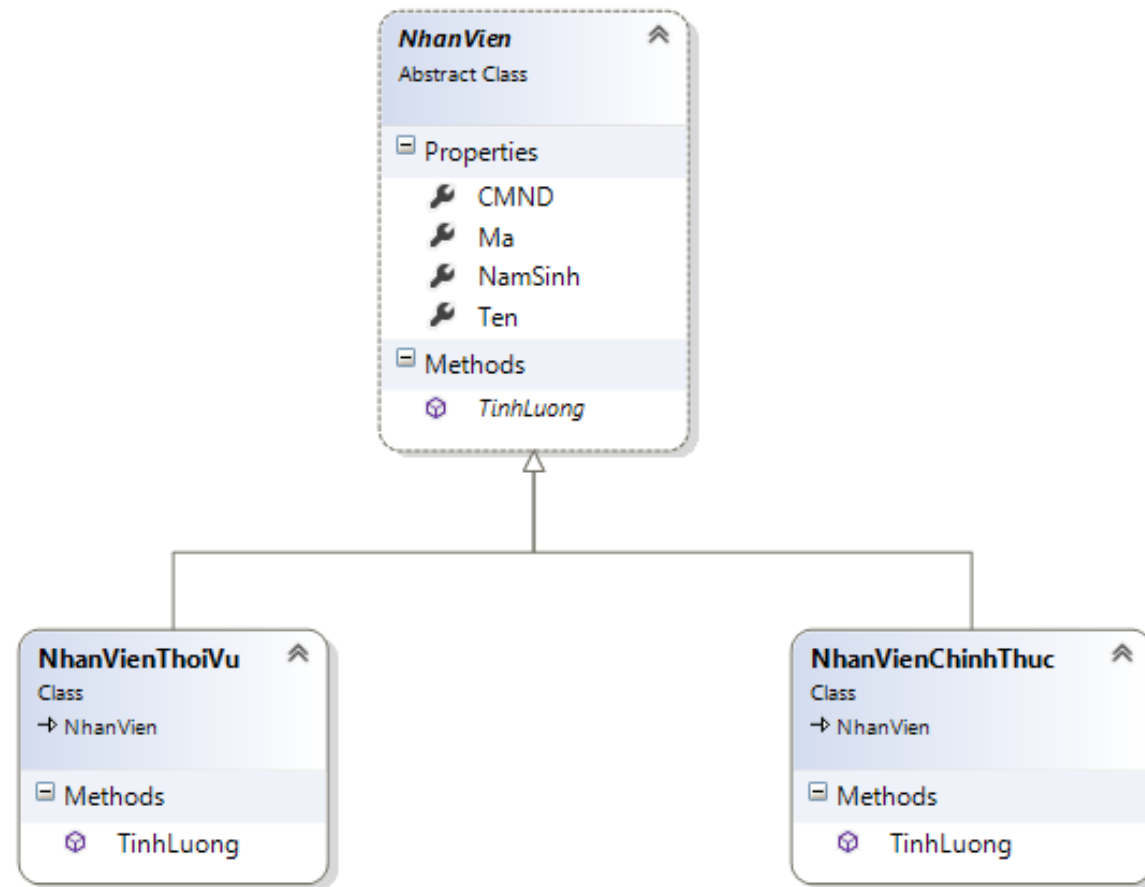


Overriding Method

- ❖ Trong một tập các lớp có mối quan hệ huyết thống có các phương thức giống signature y xì (nội dung phương thức khác nhau)
- ❖ Overriding methods giúp lập trình viên có thể định nghĩa cách hành xử khác nhau ứng với các đối tượng khác nhau nhưng cùng sử dụng một tên phương thức.
- ❖ Ví dụ: Nhân viên chính thức và Nhân viên thời vụ đều có phương thức là Tính Lương, tuy nhiên cách thức tính lương của 2 đối tượng này sẽ khác nhau.

Overriding Method

❖ Ví dụ mô hình lớp NhanVien có phương thức tính lương.



Bài Học

Tính đa hình



Tính đa hình

- ❖ Khái niệm về đa hình
- ❖ Đa hình từ kế thừa lớp
- ❖ Đa hình từ kế thừa interface

Tính đa hình

- ❖ Đa hình là tại từng thời điểm đối tượng sẽ có các hình thái khác nhau trong những hoàn cảnh khác nhau.

Tính đa hình từ Kế thừa Lớp

```
public class NhanVien {  
    public void lanhLuong()  
    {  
        System.out.println("Nhân viên lãnh lương");  
    }  
}  
  
public class NhanVienChinhThuc extends NhanVien{  
    @Override  
    public void lanhLuong() {  
        System.out.println("Nhân Viên chính thức lãnh lương");  
    }  
}  
  
NhanVien teo=new NhanVienChinhThuc();  
teo.lanhLuong();
```



Tính đa hình từ Interface

```
public interface DoAble {  
    public void doSomething(int x);  
}
```

```
public class Work implements DoAble{  
    public void doSomething(int x) {  
        System.out.println("Do "+x);  
    }  
}
```

```
public static void main(String[] args) {  
    DoAble able=new Work();  
    able.doSomething(9);  
}
```

Bài Học

Sắp xếp danh sách đối tượng dựa vào interface có sẵn



Nội dung bài học

- ❖ Giới thiệu về interface Comparable
- ❖ Cách dùng Comparable để sắp xếp

Giới thiệu về interface Comparable

- ❖ Là interface cho phép sắp xếp các đối tượng, có phương thức trừu tượng compareTo

Cách dùng Comparable để sắp xếp

```
package communityuni.com;

public class SanPham implements Comparable<SanPham>{
    private int ma;
    private String ten;
    private double gia;
    public int compareTo(SanPham arg0) {
        if(this.ma>arg0.ma)
            return 1;
        if(this.ma<arg0.ma)
            return -1;
        return 0;
    }
}
```



Cách dùng Comparable để sắp xếp

```
import java.util.ArrayList;
import java.util.Collections;

public static void main(String[] args) {
    ArrayList<SanPham> dsSp = new
    ArrayList<SanPham>();
    SanPham coca = new SanPham(5, "Cocacola", 25);
    SanPham pepsi = new SanPham(1, "Pepsi", 28);
    SanPham redbull = new SanPham(2, "Redbull", 30);
    dsSp.add(coca);
    dsSp.add(pepsi);
    dsSp.add(redbull);
    Collections.sort(dsSp);
}
```

Bài Học

Sử dụng enum trong Java



Nội dung bài học

- ❖ Khái niệm và trường hợp sử dụng enum
- ❖ Cách khai báo và sử dụng enum
- ❖ Hiệu chỉnh mô tả cho enum

Khái niệm và trường hợp sử dụng enum

- ❖ Enum là 1 dạng đặc biệt của class, nó chủ yếu được dùng để lưu trữ tập dữ liệu dạng liệt kê có giới hạn
- ❖ Dùng Enum rất khoa học trong việc kiểm tra trạng thái đối tượng sử dụng.
- ❖ Khi nào ta biết được giới hạn các trường hợp thì mới nên dùng enum
- ❖ Ví dụ 1: Về xếp loại học tập (giỏi, khá, trung bình..)
- ❖ Ví dụ 2: Về trạng thái thành công thất bại
- ❖ Ví dụ 3: Trong mọi trường hợp thay vì dùng biến để đánh dấu giá trị, ta dùng enum sẽ hiệu quả hơn.

Cách khai báo và sử dụng enum

```
public enum XepLoai {  
    XuatSac,  
    Gioi,  
    Kha,  
    TrungBinh,  
    Yeu,  
    Kem}
```

Cách sử dụng:



Hiệu chỉnh mô tả cho enum

```
public enum XepLoai {  
    XuatSac ("Xuất sắc"),  
    Gioi ("Giỏi"),  
    Kha ("Khá"),  
    TrungBinh ("Trung Bình"),  
    Yeu ("Yếu"),  
    Kem ("Kém");  
    String msg;  
    private XepLoai(String msg) {  
        this.msg=msg;  
    }  
    public String getDescription()  
    {  
        return this.msg;  
    }  
}
```

Bài Học

Bài tập rèn luyện hướng đối tượng 1



Nội dung bài học

Xây dựng lớp **NhanVien**, biết:

- ☐ Ho, Ten, SoSP lần lượt là các thuộc tính họ, tên và số sản phẩm của nhân viên.
- ☐ Viết hàm khởi tạo NhanVien(String, String, int), hàm này sẽ khởi tạo họ, tên, số sản phẩm của nhân viên; hàm phải kiểm tra số sản phẩm là số lớn hơn hoặc bằng 0, nếu là số âm thì gán giá trị cho SoSP bằng 0.
- ☐ Viết các hàm lấy và gán giá trị cho thuộc tính của lớp (các hàm get/set).
- ☐ Viết hàm getLuong() để tính lương cho nhân viên, lương = số sản phẩm * đơn giá, với đơn giá tùy thuộc vào số sản phẩm như sau:

Nội dung bài học

Số sản phẩm	Đơn giá
1 - 199	0.5
200 - 399	0.55
400 - 599	0.6
600 trở lên	0.65

Viết hàm `LonHon(NhanVien nv2)`: hàm này trả về giá trị true khi số sản phẩm (SoSP) lớn hơn số sản phẩm của `nv2`, ngược lại trả về false

➔Viết hàm main sử dụng lớp **NhanVien** theo yêu cầu sau:

Cho người dùng nhập vào 2 nhân viên, mỗi nhân viên nhập vào họ, tên, số sản phẩm của họ. Hãy tính và xuất ra lương của từng nhân viên. So sánh và xuất ra thông báo nhân viên nào có số sản phẩm nhiều hơn và nhiều hơn bao nhiêu. Dùng 2 cách so sánh: dùng hàm `LonHon` và không dùng hàm `LonHon`.

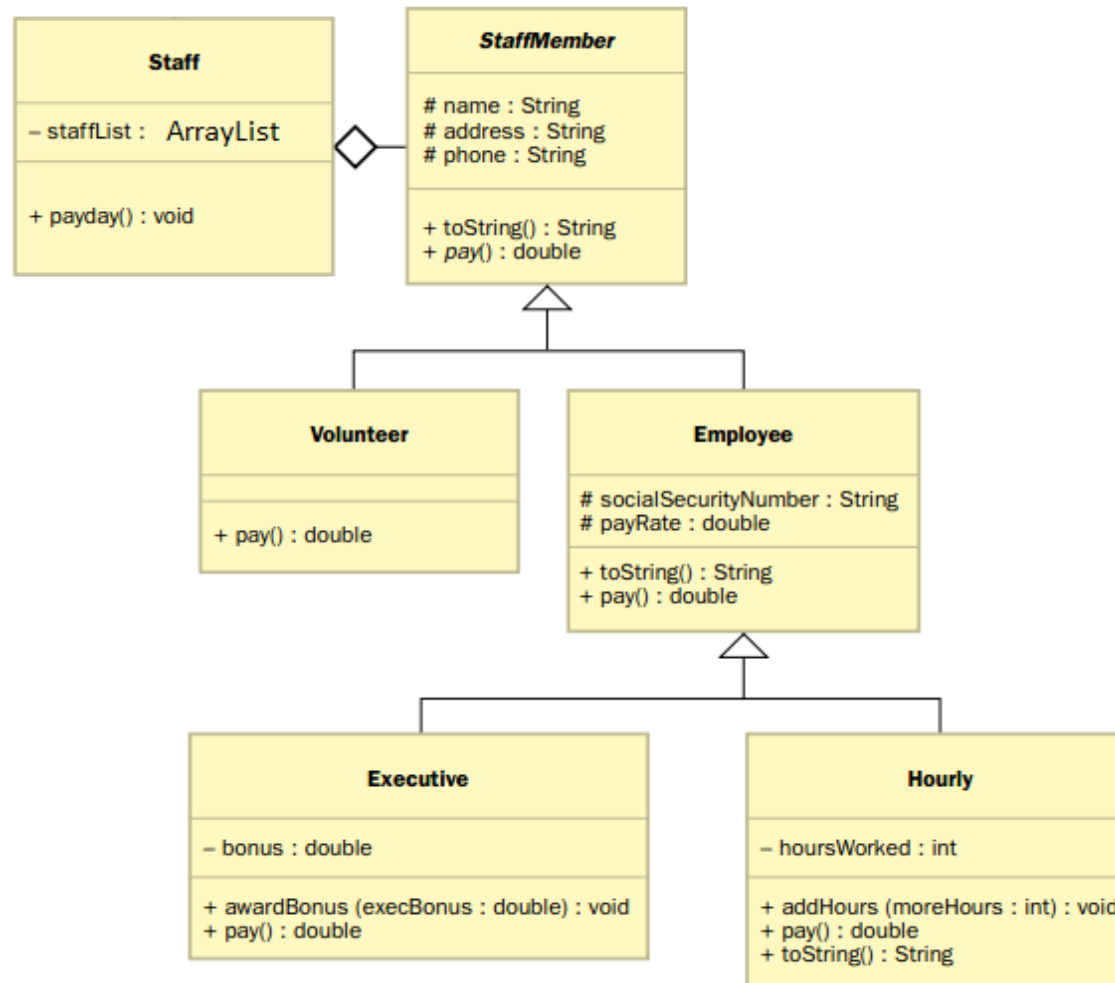
Bài Học

Bài tập rèn luyện hướng đối tượng 2



Nội dung bài học

❖ Hãy xây dựng chương trình theo mô hình lớp:



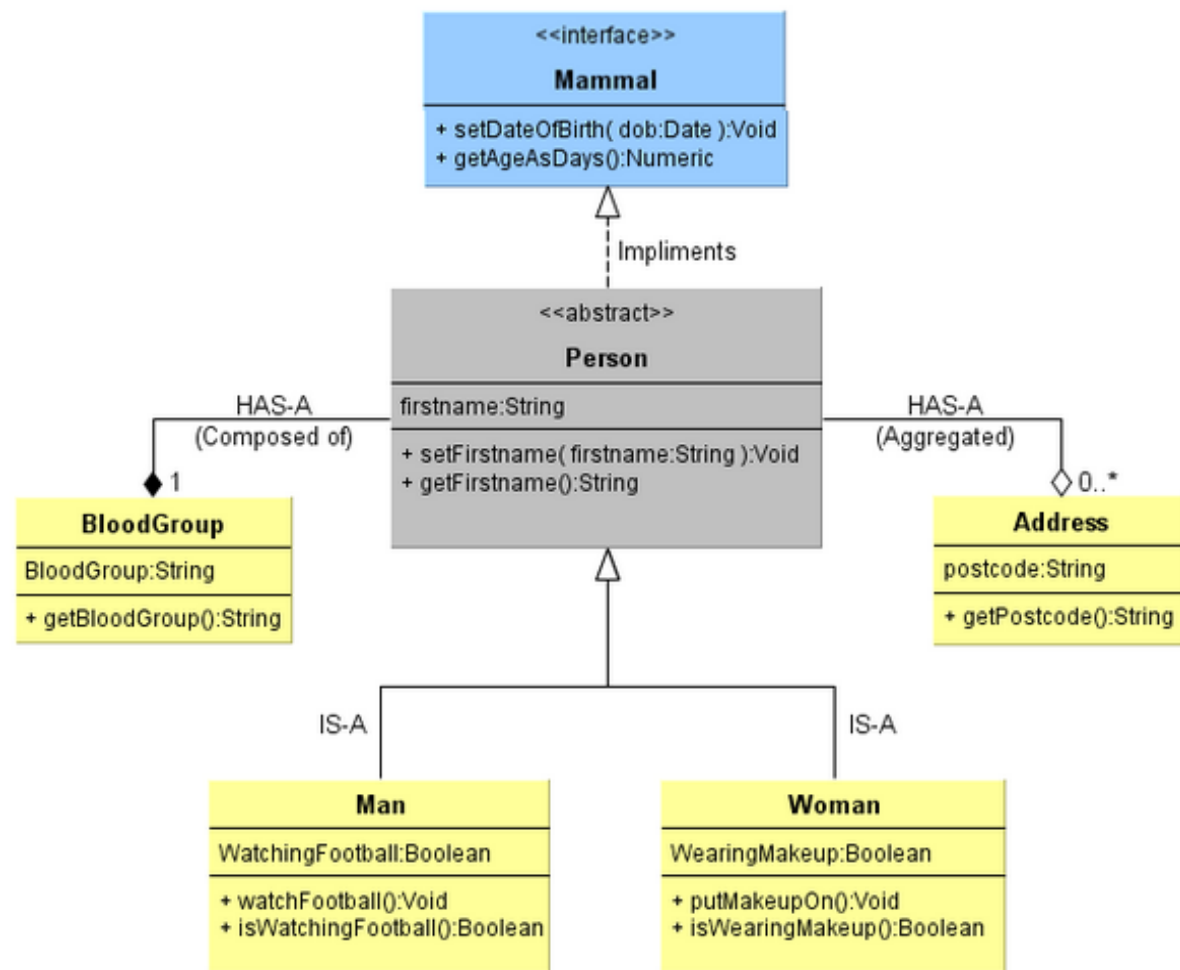
Bài Học

Bài tập rèn luyện hướng đối tượng 3



Nội dung bài học

❖ Xây dựng chương trình theo mô hình dưới đây



END

